



CS-E4960 - Software Testing and Quality Assurance D, Lecture, 3.9.2024-26.11.2024

This course space end date is set to 26.11.2024 [Search Courses: CS-E4960](#)

- ? Assignments Attendances External tools Forums Group choices Questionnaires Quizzes Resources

Syllabus

Course feedback

/ Department of Computer Science / Sections / Individual assignments / Acceptance testing 3: Oscar

Acceptance testing 3: Oscar

To do: Make a submission

To do: Receive a grade

Overview

In this assignment, you will write some more acceptance tests using Robot Framework. This time, however, you will get to work in a more realistic environment compared to the previous acceptance testing assignments.

System Under Test (SUT)

The system under test for this assignment is the Oscar Storefront that you tested manually in earlier assignments. If you don't have it running locally, you can install and run it on your machine. You can either use the local version or the Docker version for your setup.

Installation Instructions for Oscar Store Sandbox

You can follow the easy steps to install the Oscar Store sandbox locally by visiting the official documentation: <https://django-oscar.readthedocs.io/en/latest/internals/sandbox.html>

The sandbox is a minimal implementation of Oscar, designed for exploring its functionality and for developing new features. It provides a blank canvas with minimal customization, allowing you to easily experiment with the system.

- **Local Version:** The sandbox can be set up using a Python virtual environment. This will allow you to interact with Oscar locally, and explore its features in a development environment.
- **Docker Version:** If you prefer to run Oscar using Docker, there are straightforward Docker commands provided to quickly get a sandbox instance in a containerized environment.

You can access the Oscar storefront at <http://localhost:8000> once the sandbox is running with the following command:

```
docker run -p 8000:8000/tcp oscarcommerce/django-oscar-sandbox:latest
```

Keep in mind that this command is slightly different from the one provided on the website, which runs the container on port 8080.

Warning for Local Version

If you are using the local version of the Oscar sandbox, you may see that the Django Debug Toolbar is enabled by default on the website. This toolbar might hinder your acceptance tests because it can cover some of the elements on the page. To disable the toolbar, you can open the `settings.py` file located in the sandbox folder inside the repository. Then, set the `INTERNAL_IPS` list to an empty list like this:

```
INTERNAL_IPS = []
```

Part 1: Create a Test from a User Story

Below, you can find a user story and a corresponding scenario related to the Oscar application. Write an acceptance test for this user story using Robot Framework.

We have provided you with a `browse.robot` file that contains some tests and pre-defined keywords. You should add your test to this file.

Many of the elements in the Oscar source code do not have IDs, which will make accessing them more challenging compared to previous examples. You can find more information on how to locate elements using Robot Framework SeleniumLibrary [here](#).

User story and scenario

As a web store customer, I want to be able to search for a specific book, add it to my basket, and verify that I can proceed to the checkout page. Additionally, I want to be able to access the review section and submit a review.

Scenario 1: Searching for a book and adding it to the basket

- **Stimulus:** The user enters "Test Driven Development" into the search input field.
- **Response:** The page shows a list of search results with the title "Products matching "Test Driven Development"".

- **Stimulus:** The user clicks on the link for "Test-Driven Development".
- **Response:** The user is redirected to the product page for "Test-Driven Development".

- **Stimulus:** The user clicks the "Add to basket" button.
- **Response:** A confirmation message appears indicating that "Test-Driven Development" has been added to the basket.

Scenario 2: Proceeding to checkout page

- **Stimulus:** The user clicks the "View Basket" button.
- **Response:** A drop-down menu appears, showing the basket product list along with two buttons: "View Basket" and "Checkout".

- **Stimulus:** The user clicks the "Checkout" button.
- **Response:** The checkout page is displayed, prompting the user to log in or register.

- **Stimulus:** The user selects the option "I am a returning customer, and my password is".
- **Response:** The login form appears with fields to enter the password.

- **Stimulus:** The user enters the email (superuser@example.com) and password (testing) in the login form.
- **Response:** The user is logged in and is redirected to the address form page.

- **Stimulus:** The user fills out the address form.
- **Response:** The address form is successfully filled, and the user can click the "Continue" button.
- *Note:* Keep in mind that the postcode and country should be consistent (e.g., a valid US postcode should be used if "United States" is selected as the country).

- **Stimulus:** The user clicks the "Continue" button after entering the shipping details.
- **Response:** The payment page is displayed.

- **Stimulus:** The user clicks the "Continue" button on the payment page without entering any payment information.
- **Response:** The preview order page is displayed, showing a summary of the order with a "Place Order" button.

Scenario 3: Writing a review

- **Stimulus:** The user navigates back to the product page for "Test-Driven Development".
- **Response:** The product page is displayed.

- **Stimulus:** The user checks if a review by the logged-in user exists.
- **Response:**
 - If no review exists, the page shows an option to "Write a review".
 - If a review by the user already exists, the review is displayed, but the option to write another review or remove the existing review is not shown.

- **Stimulus (if no review exists):** The user clicks on the "Write a review" button.
- **Response:** A review form is displayed, allowing the user to enter a title, rating, and review text.

- **Stimulus:** The user clicks on the star rating field to select a rating.
- **Response:** The rating is visually selected, indicating the chosen number of stars.
- *Note:* Ensure that the class name for the div element changes according to the selected score (e.g., class="star-rating Four"). Verify that the corresponding stars are highlighted correctly based on the selection.

- **Stimulus:** The user enters a review title and body in the respective fields.
- **Response:** The title and body text appear in the respective fields, indicating that the user has input their review.

- **Stimulus:** The user submits the review.
- **Response:** The review is saved and displayed on the product page, showing the user's review.

Note: In this scenario, we expect you to write a review if no review by the user exists and validate if a review has been successfully created. You can verify the existence of a review or the creation of a review by checking the presence of the "Write a review" text or the actual review content on the page. Keep in mind that simply writing a review or only checking for its existence is not enough on its own; both cases must be covered.

Hints

- It can take time for elements to become visible. SeleniumLibrary has some keywords you can use to make your tests wait until an element is rendered.
- Most browsers have developer tools that allow you to inspect the page source code. This can be handy if you need to figure out the ID, name, class, or existence of some elements.
- Robot Framework also has a `Log Source` keyword, which can be used to print out the code of the SUT at the time the keyword was run.
- CSS selectors are your friend. Find out more about selecting elements using SeleniumLibrary [here](#).
- Robot Framework supports logging to console using the keyword `Log To Console`. Logging to console is a great way to debug what is going on in your tests. Additionally, each test run produces a "log.html" where you can check the results for each step of your test cases.

Part 2: Create a Test for a Problem Identified in Manual Testing

In the "Exploratory testing" assignment, you did some manual testing of the Oscar application. Using your notes from the Exploratory testing assignment, pick an issue you found.

Write a short description of this issue and explain why you think it is an issue from a user's perspective. Then, formulate a user story related to the issue and elaborate on it in a scenario. Based on the scenario, write an automated acceptance test that could be used to automatically detect this issue.

Use Robot Framework to write the test, and place the test in the `browse.robot` file where you wrote the test for part 1.

Part 3: Reflection

Write a short essay reflecting on how you experienced the relationship between exploratory testing and automated testing in the last assignments. Describe the similarities and differences between the two methods, and how they may complement each other in your view.

Additionally, think about any difficulties you may have encountered when writing tests for the Oscar storefront. What could have been done differently to avoid these difficulties, and what properties of the code helped you write tests more effectively?

A suitable length for this reflection essay is about 0.5-1 page.

Submission: Submit a .zip file (named as `studentnumber-at3.zip` replacing student number with your actual student number)that contains the following:

- The `browse.robot` file with the tests for parts 1 and 2.
- A PDF file that contains:
 - Description of the problem/situation used to create the test for part 2.
 - Your reflection essay for part 3.

Grading:

- **Part 1 (3 points):**
 - 3-2 points: The tests are clear, well-structured, and cover all three scenarios. The tests are implemented correctly, and interactions are complete with good use of keywords.
 - 2-1 points: The tests cover most scenarios with minor flaws or omissions. They are generally clear but may miss some details or optimizations.
 - 1-0 points: The tests are incomplete or unclear, missing important interactions or poorly structured.
- **Part 2 (2 points):**
 - 2-1 points: The description of the issue and the test are clear and detailed. The test covers the issue well, with good structure and sensible keywords.
 - 1-0 points: The description and test are present but may be vague or have some issues in coverage or structure.
- **Part 3 (2 points):**
 - 2-1 points: The reflection is thorough and insightful, covering the relationship between exploratory and automated testing, and discussing difficulties in writing tests.
 - 1-0 points: The reflection is limited or vague, touching on the key points but lacking depth or personal insight.

Use of AI and Collaboration with Other Students: Using AI tools to produce an answer for this assignment is not permitted. All parts of the submission must be produced by yourself (apart from files provided in the assignment). Collaboration with other students is not permitted. This is an individual assignment that you must author independently.

🔗

[browse.robot](#)

22 September 2024, 10:02 PM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	Assignment is overdue by: 120 days 3 hours
Last modified	-

Previous activity

← Acceptance Testing 2.2: Number Guessing Game

Next activity

Exploratory Testing →

Students

- MyCourses instructions for students
- Support form for students

Teachers

- MyCourses help
- MyTeaching Support

About service

- MyCourses protection of privacy
- Privacy notice
- Service description
- Accessibility summary

MyCourses support for students

Nguyen Binh (Log out)